Template Injection

Complete Security Guide — Server-Side & Client-Side • Payloads • Exploitation & Prevention

1. What is Template Injection?

Template Injection occurs when untrusted user input is embedded into a server- or client-side template without proper validation or escaping. It allows an attacker to execute arbitrary code or expressions within the context of the rendering engine.

Modern web frameworks (like Jinja2, Handlebars, Twig, or ERB) use templates to render HTML. If these engines evaluate unsanitized user input, attackers can execute code, access sensitive variables, or even achieve Remote Code Execution (RCE).

2. Types of Template Injection

- **Server-Side Template Injection (SSTI):** Input is evaluated on the server. If exploited, may lead to full system compromise depending on the template engine and its configuration.
- **Client-Side Template Injection (CSTI):** Input is evaluated by the browser (typically in JavaScript frameworks like AngularJS or Handlebars), often leading to XSS if not handled properly.

3. Recognizing Vulnerable Patterns

Look for:

- User input reflected within HTML templates or email templates
- Custom error messages, invoices, or user-generated reports
- Pages that embed variables like {{username}}, {{email}}, or {% if %} blocks
- Exposed debugging errors or stack traces with template references

Example (in Jinja2):

```
Hello {{ user_input }}
```

If user_input is {{7*7}} and the result shows 49, it's likely vulnerable.

4. Common Payloads for Testing

For SSTI (Python / Jinja2):

```
{{7*7}}  ->  49
{{'a'.__class__.__mro__[1].__subclasses__()}}  ->  Try to access system-level classes
{{config.items()}}  ->  Access app config values
{{request.args}}  ->  Dump query parameters
```

For CSTI (AngularJS, Handlebars):

```
{{constructor.constructor('alert(1)')()}}
```

or

```
{{#with "s" as |string|}}{{string.constructor("alert(1)")()}}{{/with}}
```

These test if the templating system evaluates input at runtime.

5. Impact and Severity

- **Information Disclosure:** Access to internal variables, configuration, or environment details
- **Server-Side RCE:** Exploiting insecure objects or classes
- **XSS in CSTI:** Full client-side compromise
- **SSRF or File Read:** In advanced chaining scenarios
- **Abuse in SSR apps:** Especially dangerous in Node.js, Flask, Django, Ruby apps

Severity depends on the level of control over the execution environment.

6. Real-World Examples

- **Uber (2018):** SSTI in Jinja2 allowed attackers to access environment variables containing AWS keys
- **CSTI in AngularJS-based apps:** Input fields like name={{constructor.constructor(alert(1))()}} led to DOM-based XSS
- **Twig template misuse:** Attackers could bypass logic using expressions like {{['a']|filter('system')}}

7. Techniques for Discovery

- **Brute-force payloads:** Send common injection payloads like {{7*7}}, #{7*7}, ${7*7} and check responses
- **Error-based:** Use malformed payloads to force template errors ({{['a']| badfilter}})
- **Reflection:** Inject template syntax into user-controlled input fields and see if it appears evaluated

8. How to Mitigate

For developers:

- Never trust user input in template contexts
- Escape variables properly or use strict rendering modes
- Use logic-less templating engines (e.g., Mustache) to reduce risk
- Avoid using eval or dangerous helpers (like with blocks or .constructor)
- Apply CSP (Content Security Policy) for CSTI to mitigate script execution

9. Reporting Guidelines

When reporting:

- Show the exact location and payload used
- Provide screenshots of evaluated output or proof of code execution
- Include the template engine in use (if known)
- If leading to RCE or XSS, provide full PoC and demonstrate impact
- For CSTI, explain which JS framework is in use and how the injection occurs

10. Tools for Testing

- **tplmap** – Automated SSTI exploitation tool
- **Burp Suite** – Use Intruder to fuzz template syntax
- **Custom payload lists** – Use fuzzdb or custom wordlists targeting templating engines

Conclusion

Template Injection is one of the most dangerous and often-overlooked classes of vulnerabilities. It bridges logic flaws with potential code execution and can escalate quickly. Understanding the internals of templating engines and how they're misused is key to mastering this bug class.